

SCENCE FDD v2.2

Table of Contents

SCENCE — Functional Design Document

Plataforma de gestión de campañas de influencer marketing

Versión: 2.2 | Fecha de emisión: 2026-07-03

Control de Documento

Lista de distribución

Nombre	Rol
Priscilla Perez	Founder / Product Owner
Equipo SCENCE	Operaciones / Desarrollo

Historial de cambios

Versión	Detalle del cambio	Fecha
1.0	Versión inicial del FDD	(previa a esta auditoría)
2.0	Auditoría en vivo contra producción (<code>scence-app.vercel.app</code>), 3 portales, 25 pantallas documentadas con mockups fieles, 9 bugs encontrados	2026-07-01
2.1	Reestructurado a formato ejecutivo (control de documento, mapa de proceso, requisitos funcionales por portal, reportes, no-funcionales, notificaciones, glosario). Se corrigieron 6 de los 9 bugs encontrados (ver §12, Bugs)	2026-07-01
2.1 (deploy)	Los 6 fixes + reestructuración se pushearon a producción. Un fix (eliminación de <code>CampaignDetailView.brand.tsx</code>) se basó en un diagnóstico incorrecto y rompió el build; detectado vía Vercel antes de	2026-07-01

Versión	Detalle del cambio	Fecha
	afectar usuarios, corregido y repusheado el mismo día. Se agrega hallazgo G-16 (ver Anexo C)	
2.1 (revisión Pri)	Pri revisó el FDD y reportó un problema real en BR-04 (comentario en Google Docs): tareas fantasma sin relación a deliverables. Se diagnosticó, corrigió el código (bug B-10) y se limpiaron 412 filas ya existentes en producción, con aprobación explícita de Pri para el alcance del borrado	2026-07-01
2.1 (flujo open)	Pri pidió auditar de punta a punta el flujo de campaña pública/open (crear → ver → postular → aprobar → deliverables → tareas → email → visibilidad). Se encontró bug B-11 (faltaba sync de tareas + email al aprobar) y se corrigió, acotado a <code>PATCH /api/brand/campaigns/[id]/applications</code> . Resto del flujo confirmado correcto sin cambios	2026-07-01
2.1 (roles)	Pri confirmó por comentario en el FDD que <code>agency_manager</code> no es un rol real (“no existe”). Se auditó su uso real: solo 2 perfiles de prueba, 5 RLS policies, 10 archivos de código, y el trigger de signup lo asignaba por defecto (aunque <code>ensureOrg()</code> ya lo sobrescribía a <code>brand_manager</code> en casi todos los casos). Se reasignaron los 2 perfiles a <code>super_admin</code> , se corrigió el trigger, se actualizaron las 5 RLS y los 10 archivos, sin dropear el valor del enum en Postgres (innecesario). Modelo de roles vigente: <code>super_admin</code> / <code>brand_manager</code> / <code>influencer</code>	2026-07-01
2.2	Pri pidió con urgencia mejorar el módulo de Campañas (auditoría primero, sin arquitectura nueva): filtro y columna público/privado (<code>campaigns.visibility</code>), filtro por marca (<code>brand_id</code> + <code>campaign_brands</code>), sort global reutilizable (componente <code>SortableTH</code> , ya existente, antes solo usado en la tabla de influencers), columna “Fecha creación” ordenable, pestaña “Pendientes de aprobación”, y corrección del bug de edición de campaña (abría el formulario de creación en vez de cargar los datos reales). Mismos cambios aplicados al portal Marca (mismo componente <code>CampaignsClient.tsx</code> , ver MK-02/MK-04) y se corrigió paridad de conteos en <code>/api/brand/campaigns</code> . Además: se descubrió y resolvió que la gestión real de usuarios (B-02/B-03) ya estaba construida pero nunca montada en Admin; se conectó y se retiró el flag “soon”. Se agregaron preferencias de notificación por usuario (campañas públicas, campañas privadas, alerta de fecha de entrega) para Admin, Marca e Influencer, y se reestructuró el reporte PDF de campaña (brief arriba,	2026-07-03

Versión	Detalle del cambio	Fecha
	presupuesto oculto si es 0, bloque por influencer con % completado y fecha de contenido subido).	

Sign-off

Versión	Nombre	Rol	Fecha	Firma

1. Introducción

Este Functional Design Document (FDD) documenta el diseño funcional de SCENCE tal como existe hoy en producción, verificado mediante auditoría en vivo (sesiones reales en los 3 portales, sin modificar datos) el 2026-07-01. No es una propuesta — es un registro fiel del sistema real, sus reglas de negocio, sus bugs conocidos y las decisiones de producto pendientes.

1.1 Resumen del producto

SCENCE es una plataforma SaaS B2B de gestión de campañas de influencer marketing. Permite a agencias de marketing y marcas gestionar el ciclo completo de una campaña: búsqueda y contratación de influencers, aprobación de contenido, facturación y pago.

Producto	Plataforma multi-portal (Admin, Marca, Influencer)
URL producción	https://scence-app.vercel.app
Stack	Next.js 14 (App Router), Supabase (Postgres + Auth + RLS), Vercel
Moneda base	CLP (Peso chileno)
Base de datos	Supabase project <code>xzzbishzfyovrladcaeb</code> , Postgres 17

1.2 Objetivos

1. **Centralizar** la gestión de campañas de influencer marketing en un solo sistema.
2. **Automatizar** el flujo de trabajo: invitación → aceptación → entregables → aprobación → pago.
3. **Self-service** para marcas: crear campañas y contratar influencers sin intermediación obligatoria.
4. **Transparencia** para influencers: ver campañas, entregar contenido y ver pagos.

- 5. **Control** para el admin de SCENCE: visibilidad total, capacidad de intervención sin ser cuello de botella.
- 6. **Escalabilidad** SaaS: múltiples organizaciones/clientes sobre la misma base de código.

1.3 Impacto de arquitectura legacy

El producto migró sus rutas de un esquema sin prefijo (`/campaigns` , `/brand/campaigns` , `/dashboard`) a prefijos por portal (`admin-*` , `brand-*` , `inf-*`). Las rutas viejas siguen existiendo únicamente como alias de redirect en `src/middleware.ts` por seguridad — no son UI activa. Varios de los bugs encontrados en esta auditoría (ver Anexo A) son residuos de esa migración: links en el frontend que quedaron apuntando a la ruta legacy en vez de la nueva.

2. Requisitos de Negocio

La siguiente tabla resume las reglas de negocio activas del sistema — el equivalente a los “requisitos” de un proyecto de configuración, pero registrados aquí como reglas ya implementadas y verificadas contra el código real.

ID	Regla	Descripción
BR-01	Multi-tenancy	Todo dato está scoped por <code>organization_id</code> , reforzado en cada query server-side.
BR-02	Roles de usuario	<code>super_admin</code> → Admin (ve todo); <code>is_brand=true</code> → Marca (<code>brand_manager</code> , owner); <code>is_influencer=true</code> → Influencer (sin sub-roles). <code>src/middleware.ts</code> enforza el routing por rol vía listas <code>ADMIN_ONLY</code> / <code>BRAND_ONLY</code> / <code>INFLUENCER_ONLY</code> . Corregido 2026-07-01 : el rol <code>agency_manager</code> existía en el enum y en RLS/código (2 perfiles reales, ambos de prueba) pero no correspondía a ninguna persona del modelo de producto real; se eliminó de todo uso activo (ver changelog).
BR-03	Campañas private vs. open	<code>private</code> : la marca agrega influencers por invitación. <code>open</code> : los influencers postulan y la marca decide.
BR-04	Auto-creación de deliverables	Al aceptar invitación/postulación (o al agregar un influencer a una campaña) se crean automáticamente los <code>campaign_deliverables</code> desde la plantilla de la campaña; la campaña pasa a <code>active</code> . Cada deliverable sincroniza 1:1 una <code>influencer_task</code> vinculada (<code>deliverable_id</code>) — es la única fuente de “tareas” que debe ver el influencer. Corregido

ID	Regla	Descripción
		el 2026-07-01 (bug B-10): antes también se creaban 4 tareas genéricas sin vincular a ningún deliverable real; ver Anexo A.
BR-05	<code>deliverables_spec</code> inmutable	Una vez creada la invitación, no se modifica — es el “contrato informal”. Los <code>campaign_deliverables</code> son la fuente de verdad operacional.
BR-06	Visibilidad entre portales	La marca no ve: email/teléfono de influencers, tarifas históricas con otras marcas, campañas de otras marcas, base completa de influencers, notas internas, payroll interno. El influencer no ve: tarifas de otros influencers ni datos financieros de campaña.
BR-07	Status de campaña	Pasa a <code>active</code> con el primer influencer aceptado. No se puede invitar sobre campañas <code>completed</code> / <code>canceled</code> .
BR-08	IVA en facturación	Todas las facturas incluyen IVA 19% (Chile): <code>tax_amount = subtotal * 0.19</code> .
BR-09	Moneda CLP	Moneda por defecto en toda la UI (el schema soporta otras vía <code>currency_code</code>).
BR-10	Brand self-registration	<code>/register/brand</code> auto-crea <code>brands</code> vinculado a la organización SCENCE. Primer login dispara <code>POST /api/brand/register</code> .
BR-11	Middleware como última línea de defensa	El middleware bloquea correctamente el acceso cruzado entre portales incluso cuando el frontend expone un link/botón indebido (caso bug B-08) — pero eso no exime de limpiar la UI.

2.1 Mapa de Proceso

Mapa de Proceso SCENCE

El flujo macro del negocio: se define la campaña, se convoca al elenco de influencers (por invitación directa o postulación abierta), se produce y aprueba el contenido (con retro-alimentación si se rechaza), y se cierra el ciclo con facturación y pago — cuyo cierre alimenta la siguiente campaña.

2.2 Flujo de Sistema (arquitectura)

```

Usuario → Next.js 14 (App Router, Vercel)
|
|— middleware.ts → enforza rol (admin-*/brand-*/inf-*)
|

```

- └ Supabase Auth → sesión + user_metadata (is_brand, is_influencer)
- └ Supabase Postgres (RLS + admin client server-side)
- └ campaigns, campaign_influencers, campaign_deliverables
- └ influencers, influencer_social_profiles, brands
- └ invoices, payroll_runs, bookings, contract_templates
- └ affiliate_links, support_tickets, locations, organizations
- Integraciones externas:
 - Google Calendar (Service Account) – bookings
 - Apify – sync de métricas de Instagram
 - Resend – emails transaccionales
 - Stripe – scaffold presente, checkout no activo

2.3 Supuestos

1. El equipo de SCENCE (Admin) actúa como agencia intermediaria y como operador de la plataforma a la vez.
2. Las marcas se auto-registran y operan en modo self-service; el Admin puede intervenir en cualquier campaña.
3. Los influencers son gestionados centralmente por el roster de SCENCE (Admin), no por cada marca individualmente.
4. La moneda de referencia para toda la operación comercial es CLP.
5. El middleware de Next.js es la capa de control de acceso entre portales; el frontend debe reflejar esas restricciones en la UI, no reemplazarlas.

2.4 Restricciones

- Sin app móvil nativa.
- Cobro a marcas vía Stripe: scaffold de datos listo, sin UI de checkout activa.
- Sin firma electrónica de contratos (DocuSign).
- Sin tracking de performance post-publicación en tiempo real (views/likes).
- Sin marketplace público de influencers.
- Configuración → Usuarios (gestión de equipo real): resuelto 2026-07-03, ver AD-17/MK-10 y B-02/B-03 en Anexo A. Billing/Marcas colaboradoras de Marca: siguen pendientes de decisión de producto (ver Anexo A y §7 Datos e Importación).

3. Requisitos Funcionales por Portal

Los mockups son reconstrucciones fieles (SVG, no screenshots literales) generadas a partir de la sesión en vivo del 2026-07-01, en `docs/mockups/*.svg`.

3.1 Portal Admin

Acceso: `role: super_admin`. Rutas: `admin-*`. Equipo interno de SCENCE — acceso total a todos los datos de la plataforma.

AD-01 Dashboard

Dashboard Admin

Navegación: `admin-dash` · **API:** `GET /api/dashboard` · **Tablas:** `campaigns`, `influencers`, `brands`, `invoices`, `payroll_runs`, `campaign_deliverables`

Campo	Fuente	Para qué sirve
Campañas en curso	<code>count(campaigns) where status in (active,pending_influencers)</code>	KPI de carga operativa del mes
Influencers en roster	<code>count(influencers) where is_active=true</code>	Tamaño total del roster disponible
Marcas registradas	<code>count(brands)</code>	Tamaño de la cartera de clientes
Facturado (outbound)	<code>sum(invoices.total)</code> del mes	Ingreso reconocido
Costos recibidos	<code>sum(payroll_runs.total)</code> del mes	Costo de payroll del mes
Margen bruto	<code>facturado - costos</code>	Rentabilidad rápida
Live influencers	Placeholder (siempre 0 hoy)	Feature futura de “conectados ahora”

Regla: todo se filtra por `organization_id` del usuario.

AD-02 Lista de Campañas

Lista Campañas Admin

Navegación: `admin-campaigns` · **API:** `GET /api/campaigns` · **Tablas:** `campaigns`, `brands`, `campaign_influencers`, `campaign_deliverables`

Campo	Fuente	Para qué sirve
KPIs (activas, budget total, gastado, deliverables pend.)	Agregados server-side	Cabecera de control
Campaña / Tipo / Influencers / Progreso / Budget / Fechas / Estado	<code>campaigns.*</code> join <code>campaign_influencers</code> , <code>campaign_deliverables</code>	Fila de tabla completa
Filtros (tipo, público/privado, marca, plataforma, fecha, estado)	Query params sobre <code>/api/campaigns</code>	Búsqueda server-side

Acciones: Nueva campaña, Crear con IA (AI Campaign Builder vía Claude Haiku), abrir detalle, filtrar.

Estado: ✓ mejorado 2026-07-03 — columnas “Tipo”, “Público/Privado” y “Fecha creación” agregadas; sort de columnas reutilizable vía `SortableTH` (mismo componente que ya usaba la tabla de influencers); nueva pestaña “Pendientes de aprobación” (reutiliza el filtro por `status=pending_approval` ya existente, con contador). Ver §12 changelog v2.2.

AD-03 Nueva Campaña (wizard 4 pasos)

Nueva Campaña

Navegación: `admin-campaigns/new` — mismo componente `CampaignForm.tsx` que usa Marca (`brand-campaigns/new`), API distinta por portal.

Campo	Paso	Para qué sirve
Nombre *	1 Información	Identificador visible
Descripción	1	Brief interno
Tipo *	1	Sponsored Post / Embajador / UGC / Evento / Product Seeding / Live
Visibilidad *	1	<code>private</code> (invitación) vs <code>open</code> (postulación) — BR-03
Fechas inicio/fin	2 Budget	Ventana de ejecución
Budget + Moneda	2	Techo de gasto (CLP)
Comisión %	2	Margen de agencia sobre budget
		Specs de publicación

Campo	Paso	Para qué sirve
Plataformas / Hashtags / Menciones	3 Contenido	
Guía de contenido	3	Tono/estilo para el influencer
Deliverables (tags)	3	Plantilla de entregables sugeridos
Resumen	4 Confirmar	Revisión antes de crear (<code>status=draft</code>)

AD-04 Detalle de Campaña

Detalle Campaña Admin

Navegación: `admin-campaigns/[id]` · **Tabs:** Overview · Influencers · Deliverables · Assets
· Lugares · Facturas · Historial

Campo	Fuente	Para qué sirve
Header (nombre, estado, tipo, fechas, visibilidad)	<code>campaigns.*</code>	Identificación rápida
% Completado / % Budget usado	Derivado de <code>campaign_deliverables</code> , <code>budget_total</code>	Salud de campaña
Tab Influencers	<code>campaign_influencers</code> join <code>influencers</code>	Gestión de elenco
Tab Deliverables	<code>campaign_deliverables</code>	Aprobar/ rechazar contenido
Tab Facturas	<code>invoices</code> filtradas por <code>campaign_id</code>	Trazabilidad financiera
Tab Historial	Audit log de cambios de estado	Trazabilidad de decisiones

Reusado por Marca: `brand-campaigns/[id]` reutiliza este mismo componente (`CampaignDetail`), filtrado por permisos — ver 3.2.

AD-05 Lista de Influencers

Lista Influencers Admin

Navegación: `admin-influencers` (1.450 registros) · **Tablas:** `influencers` , `influencer_social_profiles`

Campo	Fuente	Para qué sirve
Nombre / avatar	<code>influencers.display_name</code> , <code>avatar_url</code>	Identificación
Plataformas / seguidores / engagement	<code>influencer_social_profiles</code>	Evaluación de fit
Rating	<code>influencers.rating</code> (histórico de campañas)	Calidad de colaboración
Email / teléfono	<code>influencers.email</code> , <code>phone</code>	Contacto directo (solo Admin — BR-06)
Tarifas históricas	<code>influencer_rate_cards</code>	Referencia de negociación (solo Admin — BR-06)

Estado: ✓ corregido en v2.1 — bug B-01 (link roto a `/influencers/[id]`) resuelto, ver Anexo A.

AD-06 Perfil de Influencer

Navegación: `admin-influencers/[id]` · **Tabla:** `influencers`

Ficha completa: datos de contacto, redes sociales, historial de campañas, tarifas, notas internas (solo visibles para Admin).

AD-07 Data Quality

Data Quality

Navegación: `admin-influencers/data-quality` · **API:** `GET /api/influencers/duplicates` , `POST /api/influencers/merge` , `DELETE /api/influencers/bulk-delete` , `POST /api/influencers/sync-instagram`

Campo	Fuente	Para qué sirve
Total / Activos-Inactivos / Sin Instagram	Agregados sobre <code>influencers</code>	Salud de la base
Duplicados por email / IG URL / IG @	Detección server-side por coincidencia exacta	Limpieza antes de importar
Sincronizar Instagram	Llama Apify actor	Métricas frescas

AD-08 Ranking de Influencers

Ranking Admin

Navegación: `admin-influencers/ranking`

Campo	Fuente	Para qué sirve
Seguidores / Engagement / Rating	<code>influencer_social_profiles</code> , <code>influencers.rating</code>	Ordenar por criterio de negocio
Campañas / Entregables / Cumplimiento	<code>campaign_influencers</code> , <code>campaign_deliverables</code>	Desempeño histórico real

AD-09 Lista y Detalle de Marcas

Lista Marcas Detalle Marca

Navegación: `admin-brands` , `admin-brands/[id]` · **Tabs:** Overview · Campañas · Influencers · Lugares · Billing · Acceso · Historial

Campo	Fuente	Para qué sirve
Estado (Aprobada/Pendiente/Suspendida)	<code>brands.status</code>	Control de acceso al portal marca
Industria / Contacto / Email	<code>brands.*</code>	Ficha comercial
Campañas activas / totales	<code>campaigns where brand_id</code>	Volumen de negocio
Tab Acceso	<code>auth.users</code> vinculados	Invitar/gestionar acceso
Aprobar / Suspende	<code>PATCH</code> <code>brands.status</code>	Gate de acceso al portal marca

AD-10 Bookings

Bookings

Navegación: `admin-bookings` · **API:** `lib/google-calendar.ts` (Service Account) ·

Tabla: `bookings`

Vista mensual/lista sincronizada con Google Calendar. Agenda operativa de apariciones/ eventos.

AD-11 Billing (Facturas + Payroll)

Billing

Navegación: `admin-billing` (tabs Facturas/Payroll; `admin-payroll` redirige aquí) ·

Tablas: `invoices` , `payroll_runs`

Campo	Fuente	Para qué sirve
Total facturado / cobrado / vencido	Agregados sobre <code>invoices</code>	Salud de cobranza
Payroll total	<code>sum(payroll_runs)</code>	Costo total a influencers
Tabla Facturas	<code>invoices.*</code> join <code>brands</code> , <code>campaigns</code>	Gestión de cobranza (IVA 19% — BR-08)

AD-12 Contratos

Contratos

Navegación: `admin-contracts` · **Tabla:** `contract_templates` (variables `{{primary_brand_name}}` , `{{campaign_name}}` , etc.)

Plantillas de contrato reutilizables por tipo de campaña, con variables dinámicas.

AD-13 Afiliados

Afiliados

Navegación: `admin-affiliates` · **API:** `/api/affiliates` , `/api/track/[code]` · **Tabla:** `affiliate_links`

Links activos, clicks, conversiones y revenue por link de afiliado.

AD-14 Eventos & Entradas

Eventos

Navegación: `admin-events` , `admin-events/[id]` · **API:** `/api/events` , `/api/events/[id]/tickets` , `/api/events/[id]/sales`

Módulo de venta de entradas para activaciones tipo evento: total eventos, entradas vendidas, revenue, próximos.

AD-15 Analytics

Analytics

Navegación: `admin-analytics` · **Tablas:** `invoices` , `payroll_runs` , `campaigns` , `campaign_deliverables`

Vista financiera ejecutiva: revenue total, margen promedio, budget utilizado, tasa de completion, por periodo (1/3/6/12 meses).

AD-16 Soporte

Soporte Admin

Navegación: `admin-support` (todos los tickets de la organización) · **Tabla:** `support_tickets`

Triage por prioridad (P1-P3), estado (Abierto/En progreso/Cerrado), remitente y rol.

AD-17 Configuración

Configuración Admin

Navegación: `admin-settings/*` · **Tabla:** `organizations`

Tab	Fuente	Estado
Mi perfil	<code>profiles.*</code>	✓ OK
Organización	<code>organizations.*</code>	✓ OK
Usuarios	<code>organization_members</code>	✓ Resuelto 2026-07-03 (bug B-02) — el componente de gestión (invitar, listar, cambiar rol) ya existía completo pero nunca se montaba; se conectó a <code>admin-settings/users</code> y se retiró el flag “soon”
Lugares	<code>locations</code>	✓ CRUD completo — ver G-13
Notificaciones	<code>profiles.metadata.notification_preferences</code>	✓ Nuevo 2026-07-03 — 3 toggles (campañas públicas, campañas privadas, alerta de fecha de entrega), mismo componente para Admin/Marca/Influencer. Ver §9

3.2 Portal Marca

Acceso: `user_metadata.is_brand = true` . Rutas: `brand-*` . Cliente B2B — gestiona sus propias campañas y contrata influencers del catálogo SCENCE.

Regla clave de producto: el portal Marca reutiliza la experiencia Admin filtrada por permisos (no vistas paralelas reducidas). La Marca solo ve: sus campañas, influencers relacionados a sus campañas, marcas colaboradoras relacionadas (solo el nombre). La Marca NO ve: base completa de influencers SCENCE, notas internas, payroll interno, datos privados/direcciones, datos comerciales sensibles de otras marcas (BR-06).

MK-01 Dashboard

Dashboard Marca

Navegación: `brand-dash` · **API:** `GET /api/brand/campaigns` · **Tablas:** `brands` , `campaigns` , `campaign_influencers` , `campaign_deliverables`

Campo	Fuente	Para qué sirve
Campañas activas / Influencers / Para revisar	Agregados filtrados por <code>brand_id</code>	Vista rápida operativa
Lista “Tus campañas”	<code>campaigns where brand_id = brand.id</code>	Acceso directo a cada campaña propia

Regla: solo ve campañas donde `campaigns.brand_id = brand.id` (BR-06).

MK-02 Lista de Campañas

Lista Campañas Marca

Navegación: `brand-campaigns` · **API:** `GET /api/brand/campaigns` — lista real con los mismos filtros que Admin (búsqueda, tipo, plataforma, fecha, estado), reutilizando `CampaignsClient.tsx` .

Estado: ✓ corregido en v2.1 — KPI “Total gastado” mostraba `$NaN` sin datos; ahora `$0` (fix aplicado, ver Anexo A).

Estado: ✓ mejorado 2026-07-03 — mismo componente que Admin (`CampaignsClient.tsx`), por lo que Marca recibe automáticamente filtro público/privado, columnas “Tipo”/“Fecha creación” y sort global (el filtro por marca no aplica — la marca ya ve solo sus campañas). Se corrigió además paridad de conteos en `/api/brand/campaigns` (faltaban `influencer_count` / `deliverable_count` , ahora calculados desde los mismos arrays ya traídos, sin queries nuevas).

MK-03 Nueva Campaña

Mismo wizard 4 pasos que AD-03 (`CampaignForm.tsx` compartido). **Regla:** `brand_id` y `created_by_brand_id` = `brand.id` del usuario logueado (BR-10).

MK-04 Detalle de Campaña

Mismo componente que AD-04 (`CampaignDetail`), reutilizado vía `brand-campaigns/[id]` — solo si `brand_id` = la propia. Acciones de edición solo si la marca es creadora. Datos de marcas colaboradoras (si las hubiera) se muestran solo por nombre.

Estado: ✓ corregido 2026-07-03 — bug reportado por Pri: editar una campaña abría el formulario de creación en vez de cargar los datos reales. Causa: no existía `brand-campaigns/[id]/edit/page.tsx` (404 real). Se creó la ruta reutilizando `CampaignEditForm.tsx` (mismo componente de Admin), con permiso `canEdit` ya calculado server-side (bloquea a marcas co-branded que no son creadoras) y hardening en backend para que Marca nunca pueda reasignar `brand_id` vía PATCH/PUT.

MK-05 Invitar Influencer

Invitar Influencer

Navegación: `brand-campaigns/[id]/invite` · **API:** `POST /api/brand/campaigns/[id]/invite` · **Tabla:** `campaign_influencers`

Campo	Fuente	Para qué sirve
Influencer seleccionado	Viene del catálogo (<code>?influencerId=</code>)	Define a quién se invita
Tarifa propuesta (CLP)	Input libre, opcional	Si vacío, se negocia directo
Mensaje	Input libre	Contexto/objetivo para el influencer
Deliverables (tipo, cantidad, fecha)	<code>deliverables_spec</code> (inmutable, BR-05)	Genera los <code>campaign_deliverables</code> reales al aceptar

Estado: ✓ corregido en v2.1 — el submit apuntaba a un endpoint inexistente (`/api/brand-campaigns/...` en vez de `/api/brand/campaigns/...`), mismo root-cause que B-07. Ver Anexo A.

MK-06 Postulaciones / Invitaciones

Navegación: `brand-campaigns/[id]/applications` · **API:** `GET/PATCH /api/brand/campaigns/[id]/applications`

Lista de postulaciones (campañas open) e invitaciones enviadas, con acciones Aceptar/Rechazar.

Estado: ✓ corregido en v2.1 — bug B-07, mismo root-cause que MK-05 (URL de fetch no coincidía con la ruta real de la API). Ver Anexo A.

MK-07 Catálogo de Influencers

Catálogo Marca

Navegación: `brand-influencers` — catálogo filtrado a los influencers relacionados con sus campañas (no el roster completo de 1.450, por BR-06).

Campo	Visible para Marca
Nombre / avatar / plataformas / seguidores / engagement / rating	✓
Email / teléfono / tarifas históricas con otras marcas	✗ (BR-06)
Botón “Data Quality”	✗ (ocultado en v2.1 — bug B-08, ver Anexo A)
Nombre del influencer clickable a ficha admin	✗ (ocultado en v2.1 — bug B-01, no hay ficha propia de marca aún — gap G-09)

MK-08 Ranking de Influencers

Navegación: `brand-influencers/ranking` — solo relacionados a sus campañas.

MK-09 Perfil de Marca

Perfil Marca

Navegación: `brand-profile` · **Tabla:** `brands`

Campo	Fuente	Para qué sirve
Nombre empresa / RUT / Industria	<code>brands.name</code> , <code>tax_id</code> , <code>industry</code>	Ficha comercial editable por la propia marca
Sitio web / Instagram	<code>brands.website</code> , <code>instagram_handle</code>	Presencia digital
Dirección principal	<code>brands.address_*</code>	Facturación/logística — privado, no visible a otras marcas

MK-10 Configuración

Navegación: `brand-settings/*` — mismo patrón que Admin (Mi perfil, Organización, Usuarios, Lugares). Usuarios: gestión real (invitar, listar, desactivar) vía `BrandMembersSection` + `/api/brand/members`, activa desde antes de esta auditoría (ver B-03).

MK-11 Soporte

Navegación: `brand-support` — solo tickets propios.

MK-12 Billing (Marca) — (pendiente) pendiente

Navegación: `brand-billing` — marcado “soon” en v2.1 (antes 404 real — bug B-04).

Requiere decisión de producto: qué facturas/datos financieros mostrar antes de construir la vista real.

MK-13 Marcas colaboradoras — (pendiente) pendiente

Navegación: `brand-brands` — marcado “soon” en v2.1 (antes 404 real — bug B-05).

Requiere decisión de producto: alcance de “solo nombre” de otras marcas por campaña compartida (BR-06).

3.3 Portal Influencer

Acceso: `user_metadata.is_influencer = true` . Rutas: `inf-*` . Creador de contenido — ve invitaciones/campañas, sube contenido, ve historial de pagos.

IN-01 Dashboard

Dashboard Influencer

Navegación: `inf-dash` · **API:** `GET /api/influencer/campaigns` , `GET /api/influencer/tasks` · **Tablas:** `campaign_influencers` , `campaign_deliverables` , `bookings`

Campo	Fuente	Para qué sirve
Tareas pendientes / Campañas activas	Agregados sobre <code>campaign_deliverables</code> , <code>campaign_influencers</code>	Carga de trabajo
Por cobrar / Cobrado	<code>payroll_runs</code> filtrado por influencer	Transparencia de pagos
Avance de campañas	<code>deliverables aprobados / total</code>	Seguimiento de cumplimiento

IN-02 Entregables (Mis Tareas)

Entregables

Navegación: `inf-tasks` · **API:** `GET /api/influencer/tasks` , `PATCH /api/influencer/tasks/[id]` , `POST /api/influencer/deliverables/[id]/submit` · **Tabla:** `influencer_tasks` (no `campaign_deliverables` directamente — corregido en v2.1, error de documentación previo)

`influencer_tasks` es una tabla genérica de checklist personal (`source_type: campaign | booking | event | manual` , `status: pending | in_progress | done | skipped`), con un campo `deliverable_id` que la vincula 1:1 a un `campaign_deliverables` real cuando la tarea viene de una campaña (BR-04). También se usa para tareas sin deliverable (confirmar asistencia a bookings/eventos vía `createInfluencerTasks()` en `lib/influencer-tasks.ts`) — eso sí es correcto y no se tocó.

Campo	Fuente	Para qué sirve
Campaña / tipo de entregable	<code>campaign_deliverables.campaign_id, type</code>	Contexto de la entrega
Estado (Pendiente/En revisión/Aprobado/Rechazado)	<code>campaign_deliverables.status</code>	Ciclo de vida (ver §5)
Subir	Form de URL + notas	Único punto de entrega de contenido

Regla: el influencer solo ve sus propios deliverables (`WHERE influencer_id = auth.uid()`).

Estado: ✓ corregido en v2.1 — link a detalle de campaña apuntaba a ruta legacy (bug B-09, ver Anexo A).

IN-03 Campañas

Campañas Influencer

Navegación: `inf-campaigns` · **API:** `GET /api/influencer/campaigns` , `GET /api/influencer/my-campaigns`

Campo	Fuente	Para qué sirve
Asignadas por agencia	<code>campaign_influencers</code> donde <code>application_status=accepted</code>	Campañas reales de SCENCE/ marca
“Campañas propias” (botón Nueva campaña)	Tabla propia, sin marca asociada	Posible tracking personal — G-11, requiere decisión de producto

Estado: ✓ corregido en v2.1 — 2 links a detalle apuntaban a ruta legacy (bug B-09, ver Anexo A).

IN-04 Detalle de Campaña

Navegación: `inf-campaign/[id]` — reutiliza `CampaignDetailView.tsx` (mismo componente base, no el `CampaignDetail` de Admin).

IN-05 Bookings

Bookings Influencer

Navegación: `inf-bookings` · **Tabla:** `bookings`

Estado: ✓ corregido en v2.1 — bug B-06. La query pedía un join directo `bookings → brands` que no existe en el schema real (`bookings` no tiene FK a `brands`); la relación correcta es `bookings.campaign_id → campaigns.brand_id → brands.id` . Corregido anidando `brand` dentro de `campaign` en el select y aplanándolo de vuelta en la respuesta. Ver Anexo A.

IN-06 Perfil

Perfil Influencer

Navegación: `inf-profile` · **Tabla:** `influencers`

Campo	Fuente	Para qué sirve
Nombre, email, teléfono, dirección	<code>influencers.*</code>	Contacto — solo visible para el propio influencer y Admin (BR-06)
Categorías (tags)	<code>influencers.categories</code>	Filtros del catálogo de marca
Redes sociales	<code>influencer_social_profiles</code>	Base del ranking y tarifas sugeridas

IN-07 Soporte

Navegación: `inf-support` — solo tickets propios.

4. Reportes

Los siguientes reportes/vistas agregadas existen hoy en producción. A diferencia de un reporting engine dedicado, en SCENCE cada uno vive embebido en su módulo (no hay un “Report Manager” separado).

REP-01 Reporte PDF de Campaña

Propósito	Resumen ejecutivo exportable de una campaña: elenco, deliverables, budget, timeline.
------------------	--

Frecuencia de uso	Bajo demanda, al cierre o para presentar a stakeholders.
Navegación	admin-campaigns/[id]/report (Admin) · brand-campaigns/[id]/report (Marca)
Campos	Reestructurado 2026-07-03 (a pedido de Pri): brief/lineamientos de contenido arriba de todo; presupuesto oculto si es 0 o no definido; bloque por influencer (nombre, red social, seguidores, % completado, fecha de contenido subido) con sus deliverables etiquetados “Reel: URL”, “Story 1: URL”, “Story 2: URL” (numerado solo si hay más de uno del mismo tipo). Reemplaza las 2 tablas separadas de influencers y deliverables que existían antes.

REP-02 Analytics Ejecutivo

Propósito	Vista financiera de alto nivel: revenue, margen, budget utilizado, tasa de completion.
Frecuencia de uso	Mensual/trimestral.
Navegación	admin-analytics
Campos	Revenue total, margen promedio, % budget utilizado, tasa de completion, serie Revenue vs. Payroll por periodo (1/3/6/12 meses).

REP-03 Ranking de Influencers

Propósito	Ordenar el roster (o el catálogo filtrado de una marca) por desempeño real, no solo por métricas de redes.
Frecuencia de uso	Continuo, al elegir influencers para una nueva campaña.
Navegación	admin-influencers/ranking · brand-influencers/ranking
Campos	Seguidores, engagement, rating, campañas realizadas, entregables, % de cumplimiento.

REP-04 Data Quality del Roster

Propósito	Detectar duplicados e inconsistencias antes de escalar la base de influencers.
Frecuencia de uso	Mensual o antes de una importación masiva.

Navegación	<code>admin-influencers/data-quality</code>
Campos	Total influencers, activos/inactivos, sin Instagram, duplicados por email/URL/@ de Instagram.

REP-05 Billing (Facturas + Payroll)

Propósito	Salud de cobranza (facturas a marcas) y costo total de payroll a influencers.
Frecuencia de uso	Continuo/semanal.
Navegación	<code>admin-billing</code>
Campos	Total facturado, cobrado, vencido; total payroll; detalle por factura (cliente, campaña, monto, IVA, estado).

5. Convenciones de Estado y Color (Theming)

SCENCE no usa temas visuales sobre planos (no aplica, a diferencia de un sistema de gestión de espacios físicos) — el equivalente funcional es la convención de color/badge por estado de cada entidad, consistente en toda la UI:

Entidad	Estado	Color/Badge
Campaña	<code>active</code>	Verde
Campaña	<code>draft</code>	Gris
Campaña	<code>completed</code>	Violeta
Campaña	<code>canceled</code>	Rojo
Campaña	<code>paused</code>	Ámbar
Postulación/Invitación	<code>pending</code>	Ámbar
Postulación/Invitación	<code>accepted</code>	Verde
Postulación/Invitación	<code>rejected</code>	Rojo
Postulación/Invitación	<code>expired</code> / <code>withdrawn</code>	Gris
Deliverable	<code>pending</code>	Gris
Deliverable	<code>in_review</code>	Ámbar

Entidad	Estado	Color/Badge
Deliverable	approved / published	Verde
Deliverable	rejected	Rojo
Factura	paid	Verde
Factura	overdue	Rojo
Factura	sent	Ámbar
Factura	draft / void	Gris

6. Requisitos de Integración

ID	Integración	Descripción	Dirección
INT-01	Google Calendar (Service Account)	Sincroniza bookings con eventos de calendario para la agenda operativa de Admin.	Entrada/ Salida
INT-02	Apify (Instagram Sync)	Actualiza followers/engagement de influencer_social_profiles bajo demanda desde Data Quality.	Entrada
INT-03	Resend	Envío de emails transaccionales (ver §9).	Salida
INT-04	Stripe	Scaffold de columnas y modelo de datos presente (migración stripe_resend_columns), sin UI de checkout activa.	— (no activo)
INT-05	Supabase Auth	Autenticación y sesión; user_metadata determina el portal (is_brand , is_influencer).	Entrada/ Salida

7. Datos e Importación Masiva

A diferencia de un proyecto de migración de sistema legacy, SCENCE no migra datos de un sistema anterior — pero sí tiene un flujo real de carga masiva de datos que documentar:

Nombre: Importación masiva de influencers (CU-04) **Resumen:** Carga de influencers al roster vía archivo CSV. **Navegación:** admin-influencers → Bulk Upload **API:** POST /api/influencers/bulk **Volumen:** hasta 1.500 influencers por lote. **Reglas de negocio:** los

registros se crean con `status=draft` ; requiere revisión posterior en Data Quality (REP-04) antes de activarse en el roster operativo.

Nombre: Importación de ocupación/asignación (equivalente al patrón “Occupancy List Import” de proyectos de gestión de espacio) **Estado:** no implementado en SCENCE — no aplica, el modelo de asignación es por invitación/postulación (BR-03), no por importación batch.

8. Requisitos No-Funcionales

NFR-01 Auditoría / Trazabilidad

El detalle de campaña incluye un tab “Historial” con audit log de cambios de estado (ver AD-04). **No confirmado en esta auditoría:** si existe una tabla de audit trail genérica a nivel de plataforma (cambios en `influencers` , `brands` , etc.) — queda como pendiente de verificación, no como hallazgo negativo.

NFR-02 Infraestructura

Hosting en Vercel (frontend/API routes) + Supabase Cloud (Postgres, Auth, Storage). Sin infraestructura propia que mantener.

NFR-03 Rendimiento

No hay SLA de performance formalizado ni monitoreo de latencia documentado en esta auditoría.

NFR-04 Backup y Recuperación

Backups gestionados por Supabase (point-in-time recovery según plan contratado). No verificado el plan/retención exacta en esta auditoría.

NFR-05 Seguridad multi-portal

El middleware (`src/middleware.ts`) es la capa de control de acceso entre los 3 portales (BR-02, BR-11). Confirmado por auditoría que bloquea correctamente accesos cruzados incluso cuando la UI expone un botón indebido (bug B-08).

NFR-06 Licenciamiento

Stack basado en servicios de suscripción: Vercel, Supabase, Apify, Resend, Anthropic (AI Campaign Builder). Sin licencias perpetuas que gestionar.

NFR-07 Otro

N/A.

9. Notificaciones por Email

Sistema de emails transaccionales vía Resend (`src/lib/resend.ts`). Plantillas confirmadas en el código:

EMAIL-01 — Invitación a campaña (`influencerInviteEmail`) Asunto implícito: invitación de marca a campaña. Incluye nombre del influencer, marca, campaña, mensaje opcional y link a la invitación. **Estado:** ✓ **Corregido 2026-07-01 (gap G-08):** ahora se invoca desde `POST /api/brand/campaigns/[id]/invite` justo después de crear la invitación (`campaign_influencers`). No bloqueante — si el influencer no tiene email o el envío falla, la invitación queda creada igual.

EMAIL-02 — Confirmación de booking (`bookingConfirmEmail` en `resend.ts`) **Estado:** ✓ **Corregido 2026-07-01 (gaps G-08 y G-14):** se confirmó que no era una duplicación real sino dos pasos de un mismo flujo: `bookingConfirmationEmail` (inline, en `send-confirmations/route.ts`) pide al influencer que confirme/decline vía botones; `bookingConfirmEmail` (`resend.ts`) ahora se invoca como recibo de confirmación en `GET /api/bookings/confirm` cuando el influencer hace clic en “confirmar”. Flujo completo: Admin dispara `send-confirmations` desde `BookingsClient.tsx` → influencer confirma → recibe recibo.

EMAIL-03 — Estado de deliverable (`deliverableStatusEmail`) **Estado:** ✓ activa, invocada desde `src/app/api/emails/deliverable-status/route.ts` . Notifica al influencer cuando su entrega es aprobada o rechazada, con notas de revisión si aplica.

EMAIL-04 — Factura (`invoiceEmail`) **Estado:** ✓ activa, invocada desde `src/app/api/invoices/[id]/route.ts` . Notifica a la marca el detalle de una factura (cliente, total, vencimiento).

Preferencias de notificación por usuario — Nuevo 2026-07-03. Distinto de las plantillas transaccionales de arriba: son 3 toggles on/off por usuario, guardados en `profiles.metadata.notification_preferences` (sin tabla nueva), vía `PATCH /api/settings/profile` : campañas públicas (email al publicarse una abierta), campañas privadas (email al ser invitado/asignado), y alerta de fecha de entrega próxima. Mismo componente (`NotificationPreferencesForm.tsx`) en Admin (`admin-settings/notifications`), Marca (`brand-settings/notifications`) e Influencer (sección en `inf-profile`). Por ahora solo persiste la preferencia — el envío real de emails/alertas conectado a estos toggles queda como fase posterior.

10. Documentos de Apoyo

Documento	Descripción	Ubicación
Este FDD (fuente)	Versión Markdown, control de versiones	<code>docs/</code> <code>FUNCTIONAL_DESIGN.md</code> (repo)
Este FDD (Word)	Copia editable/imprimible	Carpeta del proyecto
Mockups de pantallas	29 SVG reconstruidos de la auditoría en vivo	<code>docs/mockups/*.svg</code>
Migraciones de base de datos	Definición real de tablas (con drift conocido — ver nota)	<code>supabase/migrations/</code> <code>*.sql</code>
Middleware de rutas	Enforcement de roles por portal	<code>src/middleware.ts</code>

Nota de integridad: durante esta auditoría se confirmó que las migraciones en el repo **no reflejan el 100% del schema real** de producción (ej. la tabla `brands` no tiene migración de creación en el repo, pero existe y está en uso — fue creada fuera del flujo de migraciones versionadas). Se recomienda una tarea futura de reconciliación schema-real vs. migraciones.

11. Glosario

Término	Definición
BR	Business Rule — regla de negocio (§2)
RLS	Row Level Security — políticas de acceso a nivel de fila en Postgres/Supabase
Deliverable	Entregable de contenido asociado a una campaña e influencer
Booking	Reserva/evento agendado con un influencer (aparición, shoot, evento)
Payroll run	Corrida de pago agrupada a uno o más influencers
CLP	Peso chileno — moneda base de la plataforma
IVA	Impuesto al Valor Agregado (19% en Chile) — BR-08
<code>is_brand</code> / <code>is_influencer</code>	Flags en <code>user_metadata</code> de Supabase Auth que determinan el portal del usuario
<code>organization_id</code>	

Término	Definición
	Identificador de tenant — todo dato queda scoped a una organización (BR-01)
<code>application_status</code>	Estado del ciclo invitación/postulación (<code>pending</code> , <code>accepted</code> , <code>rejected</code> , etc.)
Campaña <code>private</code> vs <code>open</code>	Visibilidad de campaña: por invitación directa vs. postulación abierta (BR-03)
Portal	Uno de los 3 frontends de la app: Admin, Marca, Influencer

12. Anexo A — Bugs encontrados y su estado

Se encontraron 9 bugs de producción en la auditoría en vivo del 2026-07-01. **6 fueron corregidos** con commits pequeños e independientes (sin push a producción, pendiente de aprobación de deploy). 4 quedaron marcados “(pendiente) soon” en vez de construir la feature completa, por requerir decisión de producto sobre qué datos mostrar (relacionado a BR-06).

ID	Severidad	Portal	Descripción	Estado v2.1
B-01	● Crítica	Admin/ Marca	Clic en influencer del roster → 404 (<code>/influencers/[id]</code> no existe)	✓ Corregido — link real a <code>/admin-influencers/[id]</code> ; en Marca, no clickable (sin ficha propia aún)
B-02	● Media	Admin	Configuración → Usuarios mostraba el formulario de Organización (alias engañoso)	✓ Resuelto 2026-07-03 — se descubrió que el componente de gestión (TeamMembers: invitar, listar, cambiar rol) ya existía completo en el código pero nunca se montaba en ninguna página. Se conectó a admin-settings/users y se retiró el flag “soon”. Ver AD-17
B-03	● Media	Marca	Mismo bug que B-02	✓ Resuelto — a diferencia de B-02, Marca ya contaba con gestión real de usuarios (BrandMembersSection + <code>/api/brand/members</code> : invitar, listar, desactivar), sin flag “soon”. Confirmado en la revisión del 2026-07-03. Ver MK-10
B-04	● Alta	Marca	<code>brand-billing</code> → 404	✓ Mitigado — nav marcado “soon” en vez de 404. Construir la vista real queda

ID	Severidad	Portal	Descripción	Estado v2.1
				pendiente de decisión de producto (qué facturas mostrar)
B-05	● Alta	Marca	<code>brand-brands</code> → 404	✓ Mitigado — mismo tratamiento. Requiere definir alcance de “solo nombre” de marcas colaboradoras (BR-06)
B-06	● Media	Influencer	Error crudo de Supabase: <i>“Could not find a relationship between ‘bookings’ and ‘brands’”</i>	✓ Corregido — join reescrito vía <code>bookings.campaign_id → campaigns.brand_id → brands.id</code>
B-07	● Media	Marca	Tab de postulaciones/ invitaciones: <code>Unexpected token '<' ... is not valid JSON</code>	✓ Corregido — el fetch apuntaba a <code>/api/brand-campaigns/...</code> en vez de <code>/api/brand/campaigns/...</code> . Se encontró y corrigió el mismo bug en el submit de “Invitar influencer” (no reportado antes)
B-08	● Baja	Marca	Botón “Data Quality” visible, apunta a herramienta admin-only	✓ Corregido — oculto para Marca
B-09	● Alta	Influencer	Clic en campaña asignada → 404 (<code>/campaign/[id]</code> legacy)	✓ Corregido — 4 ocurrencias actualizadas a <code>/inf-campaign/[id]</code>
B-10	● Media	Influencer	“Tareas pendientes” (<code>inf-dash</code> , <code>inf-tasks</code>) mostraba tareas fantasma sin relación a ningún deliverable real, mezcladas con las reales	✓ Corregido — reportado por Pri revisando el FDD (BR-04). <code>POST /api/campaigns/[id]/influencers</code> creaba 4 <code>influencer_tasks</code> genéricas hardcodeadas (<code>createInfluencerTasks</code>) además de las sincronizadas 1:1 con <code>campaign_deliverables</code> (<code>syncDeliverableTask</code>). Se quitó la llamada genérica en ese flujo (no se tocó su uso legítimo en bookings/eventos) y se borraron 412 filas fantasma ya existentes en producción (348 sin interacción + 64 marcadas “done”)
—		Marca		✓ Corregido — guard <code>?? 0</code> agregado

ID	Severidad	Portal	Descripción	Estado v2.1
	● Muy baja		KPI “Total gastado” mostraba \$NaN sin datos	
B-11	● Media	Influencer	Al aprobar una postulación a campaña open (<code>PATCH /api/brand/campaigns/[id]/applications</code> , <code>action=accept</code>), se creaban los <code>campaign_deliverables</code> reales pero nunca se llamaba a <code>syncDeliverableTask</code> — el influencer no veía la tarea en “Mis tareas” pese a tener un deliverable real asignado. Es el gap inverso a B-10 (faltaban tareas reales, no sobraban fantasmas). Tampoco se enviaba ningún email al aprobar	✓ Corregido — encontrado en auditoría del flujo open/postulación pedida por Pri (2026-07-01). Se agregó el <code>syncDeliverableTask</code> por cada deliverable creado y un email de aprobación (<code>campaignApplicationApprovedEmail</code> , plantilla nueva en <code>resend.ts</code> , no reutiliza <code>deliverableStatusEmail</code> porque ese es para estado de contenido ya enviado, no para aprobación de participación)

Incidente post-deploy (corregido el mismo día): en un primer intento se eliminó `CampaignDetailView.brand.tsx` asumiendo que era código muerto — un `grep` insuficiente (solo se miraron los nombres de archivo resultantes, no el contenido de las líneas) llevó a esa conclusión errónea. El archivo **sí tiene un import estático real** desde `CampaignDetailView.tsx` (`import { BrandCampaignView } from './CampaignDetailView.brand'`), así que borrarlo rompió la compilación de producción (deploy `dpl_CmzEp4WBZodkxNhFPD98HkJ4yCjM` → `ERROR` , detectado vía el conector de Vercel antes de que afectara a usuarios reales, ya que Vercel no promueve un build fallido al alias de producción). Se restauró el archivo, se corrigió en él el mismo bug de URL que B-07 (2 fetches), y se validó con un script que confirma 0 imports rotos (relativos y `@/`) en todo `src/` antes de repushear. Ver G-16 para el hallazgo funcional real detrás de este archivo.

Patrón común (B-01, B-04, B-05, B-09): residuos de la migración de rutas a los prefijos `admin-*/brand-*/inf-*` .

Actualización 2026-07-03: B-02 y B-03 (gestión de usuarios) quedaron resueltos de forma definitiva, no solo mitigados. Además se implementó el módulo de campañas mejorado (filtro/

columna público-privado, filtro por marca, sort global reutilizable, columna “Fecha creación”, pestaña “Pendientes de aprobación”, corrección del bug de edición que abría creación en vez de cargar datos reales) y preferencias de notificación por usuario. Ver detalle en §3.1 AD-02/AD-17, §3.2 MK-02/MK-04/MK-10, §9 y el changelog v2.2 (Control de Documento).

13. Anexo B — Diagramas de flujo detallados

B.1 Registro Marca (self-service)

flowchart TD

```
A["Marca visita /register/brand"] --> B["Completa: empresa, contacto, email, password"]
B --> C["signUp Supabase – is_brand=true, brand_name en metadata"]
C --> D["Email de confirmación enviado"]
D --> E["Click en link → /auth/callback"]
E --> F["Redirect → /brand-dash"]
F --> G["Layout detecta is_brand=true"]
G --> H["POST /api/brand/register"]
H --> I["brands: org_id = org SCENCE, user_id = auth.uid"]
I --> J["Dashboard vacío – lista para crear primera campaña"]
```

B.2 Creación de campaña

flowchart TD

```
A["/brand-campaigns/new"] --> B["Wizard: Información → Budget → Contenido → Confirmar"]
B --> C["POST /api/brand/campaigns"]
C --> D["campaigns: status=draft, brand_id, created_by_brand_id"]
D --> E["Redirect → /brand-campaigns/[id]"]
E --> F["Campaña en draft – lista para invitar influencers"]
```

B.3 Invitación influencer (campaña private)

flowchart TD

```
A["Marca → /brand-influencers"] --> B["Busca y filtra influencers"]
B --> C["Click 'Invitar' → /brand-campaigns/[id]/invite?influencerId=..."]
C --> D["GET /api/brand/influencers/[id]"]
D --> E["Completa fee, mensaje, deliverables"]
```

```

E --> F["POST /api/brand/campaigns/[id]/invite"]
F --> G[("campaign_influencers: origin=invitation,
application_status=pending")]
G --> H{"Influencer responde"}
H -->|Acepta| I["PATCH application_status=accepted"]
H -->|Rechaza| J["application_status=rejected"]
I --> K["Auto-crea campaign_deliverables desde deliverables_spec"]
K --> L["campaign.status → active (si era draft)"]

```

B.4 Postulación (campana open)

flowchart TD

```

A["Campana visibility=open"] --> B["GET /api/influencer/campaigns/open"]
B --> C["Influencer postula + mensaje"]
C --> D["POST /api/influencer/campaigns/[id]/apply"]
D --> E[("campaign_influencers: origin=application, status=pending")]
E --> F["Marca ve en /brand-campaigns/[id]/applications"]
F --> G{"Marca decide"}
G -->|Acepta| H["Auto-crea deliverables + campaign.status=active"]
G -->|Rechaza| I["application_status=rejected"]

```

B.5 Entrega y aprobación de deliverables

flowchart TD

```

A["Influencer ve /inf-tasks"] --> B["Sube URL de contenido + notas"]
B --> C["POST /api/influencer/deliverables/[id]/submit"]
C --> D["deliverable.status → in_review"]
D --> E["Marca/Admin ve 'X para revisar' en dashboard"]
E --> F["Revisa contenido (link externo)"]
F --> G{"Decisión"}
G -->|Aprueba| H["status → approved · EMAIL-03"]
G -->|Rechaza| I["status → rejected + review_notes · EMAIL-03"]
I --> B

```

B.6 Facturación

flowchart TD

```

A["Admin crea invoice en /admin-billing"] --> B["POST /api/invoices – IVA 19%"]
B --> C[("invoices: status=draft")]

```

C --> D["draft → sent → paid"]

D --> E["Marca ve la factura · EMAIL-04"]

B.7 Payroll

flowchart TD

A["Admin crea payroll run en /admin-billing (tab Payroll)"] --> B["POST /api/payroll"]

B --> C["payroll_runs: agrupa pagos a influencers"]

C --> D["pending → approved → processing → paid"]

D --> E["Influencer ve su pago en /inf-dash"]

14. Anexo C — Matriz de permisos y estados

C.1 Matriz de permisos por rol

Nota (2026-07-01): se elimina la columna `agency_manager` — no correspondía a ningún rol real del producto (ver BR-02 y changelog). Modelo vigente: `super_admin` (Admin, ve todo) | `brand_manager` (Brand, owner) | `influencer` (sin sub-roles).

Recurso/ Acción	super_admin	brand_manager (is_brand)	influencer (is_influencer)
Leer campaigns (todas)	✓	✗ — solo propias + colaboradoras por invitación (<code>campaign_brands</code>), confirmado en <code>/api/brand/campaigns</code>	✗
Crear campaign	✓	✓	△ ver G-11
Editar campaign	✓	✓ (propia, pre-activa)	✗
Borrar campaign	✓	✗	✗
Leer influencers (todos)	✓	✓ (limitado)	✗
Crear/editar influencer	✓	✗	✗
	✓	✓	✗

Recurso/ Acción	super_admin	brand_manager (is_brand)	influencer (is_influencer)
Invitar influencer			
Postular (aplicar)	✗	✗	✓
Aceptar/ rechazar postulación	✓	✓	✗
Aceptar/ rechazar invitación	✓ (forzar)	✗	✓
Subir deliverable	✗	✗	✓
Aprobar deliverable	✓	✓	✗
Leer invoices	✓	✓ (propias)	✗
Crear invoice	✓	✗	✗
Leer payroll	✓	✗	✓ (propio)
Crear payroll	✓	✗	✗
Leer brands	✓	✓ (propia)	✗
Leer analytics	✓	✗	✗
Sync Instagram	✓	✗	✗

C.2 Estados y transiciones

Campaign.status: draft → pending_influencers → active ≠ paused → completed ;
draft/active → canceled

application_status: pending → accepted / rejected / expired / withdrawn

Deliverable.status: pending → in_review → approved → published ; in_review → rejected → in_review (re-entrega)

invoice_status: draft → sent → paid ; sent → overdue → partially_paid ; draft → void

payroll_status: pending → approved → processing → paid / failed

C.3 Gaps de producto vigentes

Gap	Descripción	Impacto
G-01	Influencer no tiene UI de “Invitaciones” separada de Campañas	Medio
G-02	No hay /opportunities dedicado... Resuelto 2026-07-01 (aclarado con Pri): no era un gap real de construcción — crear campañas <code>visibility: 'open'</code> ya funciona en Admin (<code>CampaignForm.tsx</code>) y Marca (<code>CampaignFormView.brand.tsx</code> , real en <code>brand-campaigns/new</code>), y los influencers de la misma organización ya las ven en <code>inf-dash</code> (sección “Campañas Disponibles”, vía <code>/api/influencer/campaigns/open</code>) y pueden postular. Abierta = dentro de la organización, no marketplace público entre marcas. Pri confirmó que así está bien, no se necesita página dedicada	Cerrado
G-06	No hay notificaciones en tiempo real	Medio
G-07	<code>campaign_influencers.status</code> legacy convive con <code>application_status</code>	Técnico
G-08	Emails de invitación y de confirmación de booking no se invocan... Resuelto 2026-07-01: conectados ambos (ver §9, EMAIL-01/02)	Cerrado
G-09	No hay perfil público de influencer para marcas (<code>/brand-influencers/[id]</code>)	Bajo
G-10	Sin onboarding guiado para marcas nuevas	Bajo
G-11	“Campañas propias” del influencer no está en el modelo de permisos documentado	Medio — requiere decisión de producto
G-13	Configuración → Lugares es un stub... Resuelto 2026-07-01: la tabla <code>locations</code> existía (RLS incluida) pero 0 endpoints la usaban — no era “conectar UI a API existente” como se pensó al inicio, había que construir el API completo. Se construyó <code>GET/POST /api/locations</code> + <code>PATCH/</code>	Cerrado

Gap	Descripción	Impacto
	<code>DELETE /api/locations/[id]</code> (org-scoped, solo <code>super_admin</code>) y la UI (lista + modal crear/editar + borrar) en <code>admin-settings/locations</code>	
G-14	Duplicación de plantillas de email de booking... Resuelto 2026-07-01: no era duplicación, eran 2 pasos de un mismo flujo (solicitud de confirmación + recibo). Se conectó el recibo (<code>bookingConfirmEmail</code>) que estaba muerto	Cerrado
G-15	Migraciones del repo no reflejan <code>brands</code> ... Resuelto 2026-07-01: agregada migración baseline (<code>20260701000001_baseline_brands_table.sql</code>) documentando columnas, constraints, índices y RLS reales de <code>brands</code> , sin tocar producción (tabla ya existe ahí). Hallazgo nuevo durante esta tarea (no corregido, requiere aprobación aparte): las 4 RLS policies de <code>brands</code> comparan <code>organization_id</code> contra una subquery que también selecciona <code>brands.organization_id</code> (no <code>profiles.organization_id</code> , columna que no existe) — la condición es tautológica y en la práctica no filtra por organización a nivel RLS. Bajo riesgo real hoy porque las 20 rutas que tocan <code>brands</code> usan <code>createAdminClient()</code> (service role, bypassea RLS) con su propia lógica de autorización — pero es una brecha de defensa en profundidad si algo llega a consultar <code>brands</code> desde el browser con la key anon/authenticated	Cerrado (baseline). RLS de <code>brands</code> queda como hallazgo de seguridad pendiente de decisión — ver nota.
G-16	<code>CampaignDetailView.tsx</code> soporta <code>mode="brand"</code> ... Resuelto 2026-07-01: confirmado por grep que ninguna ruta real usaba <code>mode="brand"</code> (Marca usa <code>CampaignDetail</code> directo). Se eliminó la rama y el archivo <code>CampaignDetailView.brand.tsx</code> , validado con script exhaustivo de imports (0 rotos) + <code>tsc --noEmit</code> (0 errores) antes de commitear	Cerrado

Fin del documento — SCENCE FDD v2.1, 2026-07-01.